Step 1 — The Report Shape and the Compiler — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the four shapes
- Hint 2 — compile and format, with the one edge case

Hint 1 — the four shapes

Four Pydantic models in `src/schemas.py`:

- `ActionItem(task: str, owner: Optional[str] = None)` — one task, maybe an owner.
- `ActionItems(items: List[ActionItem] = [])` — the wrapper the extractor returns (a model with one list field, so `with_structured_output` has a single object to fill).
- `Decisions(decisions: List[str] = [])` — same idea for decision strings.
- `MeetingReport(summary, decisions, action_items)` — the compiled result.

Why wrap the lists? `with_structured_output(Model)` wants a model, not a bare `list`. Wrapping the list in a one-field model is the standard way.

Hint 2 — compile and format, with the one edge case

`compile_report(parts)` takes the dict `RunnableParallel` will produce and builds the report:

```
compile_report(parts):
    return MeetingReport(
        summary = parts["summary"],              # a plain string
        decisions = parts["decisions"].decisions, # unwrap the Decisions model
        action_items = parts["actions"].items,    # unwrap the ActionItems model
    )
```

`format_report(report)` builds the printed block with plain `for` loops. The one edge case: an action with no owner must read **"unassigned"**, not blank:

```
owner = "unassigned"
if item.owner:
```

```
    owner = item.owner
line = f"  - {item.task} ({owner})"
```

(Use an `if`, not a ternary.) Test both with **hand-made** `Decisions` / `ActionItems` — one action with an owner, one without — and no model at all. The whole file set is in the Solution.